Reflection Journal for Text Preprocessing

Working through this lab changed the way I think about text preprocessing in Natural Language Processing. Before this lab, I thought preprocessing was mainly about cleaning unnecessary words and symbols from text. Now I understand that preprocessing is much more important because every decision made during this stage can affect how an NLP system understands language. Something that looks unnecessary to a person may actually contain useful information for a computer. One of my biggest insights was that there is no single preprocessing method that is best for every NLP application.

One concept that stood out to me was tokenization. I learned that even something as simple as dividing a sentence into words requires decisions about contractions, punctuation, emojis, and other symbols. For example, NLTK separated “It’s” into different tokens and also treated punctuation as separate tokens. This made me realise that the way text is divided can influence everything that happens afterward. Social media text makes this even more challenging because people use slang, abbreviations, hashtags, emojis, and repeated punctuation. In the lab, spaCy appeared more useful for this type of messy text because it handled these different elements more consistently.

Another important lesson for me was that removing more information does not necessarily mean better preprocessing. I initially assumed that cleaner and shorter text would always improve an NLP model. However, the stop-word comparison showed why this assumption can be dangerous. spaCy removed more words and punctuation than NLTK in the example, reducing the vocabulary by 50% compared with 28.6% for NLTK. While this can make processing more efficient, vital information may disappear. Words such as “not,” “no,” and “never” can completely change the sentiment of a sentence. This helped me understand that preprocessing should depend on what I am trying to analyse.

I also found the comparison between stemming and lemmatization interesting. Stemming seems useful when speed is important because it quickly reduces different forms of words to similar roots. However, the results can sometimes look unnatural. Lemmatization seems more intelligent because it tries to preserve the actual meaning of the word. From this comparison, I would prefer stemming for applications such as search engines where matching word variations quickly is important. For sentiment analysis or chatbots, I would prefer lemmatization because understanding meaning accurately is more important

The biggest challenge for me was deciding what information should be removed and what should be preserved. At first, punctuation, emojis, and hashtags seemed like noise. After looking at social media examples, I realised they can communicate emotion and meaning. Removing emojis can hurt sentiment analysis, while removing hashtags can remove information about topics and trends. A question that came to my mind is: How can we automatically determine the best preprocessing strategy for a particular dataset instead of choosing it manually? I also wonder how much preprocessing is still necessary when using modern language models that can already understand informal and complex text.

In future projects, I can see myself using these techniques for sentiment analysis, chatbots, search systems, and text classification. For example, if I build a system that analyses customer reviews, I would preserve words and symbols that express opinions while removing information that does not contribute much meaning. If I work with social media data, I would

be careful about removing emojis and hashtags. Overall, this lab taught me that good preprocessing is not about removing as much text as possible. It is about finding the right balance between cleaning the data and preserving the information that matters for the specific NLP task. This is a principle I expect to use when building future NLP and AI projects.